Facial Emotion Recognition: A Comparative Study of Custom CNN, ResNet-50, and Vision Transformer Architectures

MATH 8335: Deep Learning

Final Project Report

Theophilus Dwamena Frimpong

20738050

School of Mathematics and Statistical Sciences

University of Texas Rio Grande Valley

May 4, 2026

Abstract

This work compares three deep learning architectures for seven-class Facial Emotion Recognition (FER) on the Real-world Affective Faces Database (RAF-DB), comprising 15,339 crowd-annotated RGB images across seven emotion categories. A custom CNN trained from scratch, ResNet-50 fine-tuned from ImageNet-pretrained weights, and ViT-B/16 fine-tuned from ImageNet-21k-pretrained weights are implemented and evaluated under identical training conditions, so that performance differences are attributable to architectural choice alone. ResNet-50 achieves the strongest performance across all metrics (accuracy 77.18%, macro-F1 0.6939), outperforming ViT-B/16 (70.96%, 0.6041) and the custom CNN (61.54%, 0.5217), demonstrating that pretrained convolutional networks remain the most effective architecture for moderately sized and imbalanced facial expression datasets, where the spatial inductive bias of convolutions outweighs the representational advantages of global self-attention.

1 Introduction

1.1 Background and Motivation

The human face is one of the richest channels of non-verbal communication available to us. Pioneering cross-cultural research by Ekman and Friesen [1] established that at least six discrete emotional states, which include anger, disgust, fear, happiness, sadness, and surprise, are expressed through universal facial configurations recognizable across cultures and language boundaries. This biological grounding has made facial expressions a natural target for automated analysis. Facial Emotion Recognition (FER) refers to the automatic classification of such expressions from still images or video frames into discrete emotion categories. Kaur and Kumar [2] describes how FER has penetrated domains including medical diagnosis, driver monitoring systems, customer feedback analysis, and educational engagement tracking.

Despite the theoretical clarity of Ekman's model, automated FER in real-world conditions remains a challenging computer vision problem. Images collected from the internet exhibit enormous variability. The problem is further complicated by the subjectivity of emotion labeling where even trained annotators do not always agree, especially for ambiguous states such as fear versus surprise or anger versus disgust [3]. Building models that generalize reliably to such conditions remains an active area of research, and comparing different deep learning paradigms, from classical CNNs to modern Transformers, on this task provides practically relevant insight.

1.2 Problem Statement

This project addresses FER as a supervised multi-class image classification task. Formally, given an input image $\mathbf{x} \in \mathbb{R}^{H \times W \times 3}$ depicting a human face, the goal is to learn a mapping $f_\theta : \mathbf{x} \mapsto \hat{y} \in \{0, 1, \ldots, 6\}$ that assigns the image to one of seven emotion categories. Model parameters θ are optimized by minimizing a class-weighted cross-entropy loss, and performance is evaluated on a held-out test set using overall accuracy and macro-averaged F1 score. The benchmark used throughout is the Real-world Affective Faces Database (RAF-DB) [4], comprising 15,339 crowd-annotated RGB face images across seven emotion classes. Three deep learning models of increasing architectural complexity are implemented and compared: a custom CNN trained from scratch, ResNet-50 [5] fine-tuned from ImageNet-pretrained weights, and ViT-B/16 [6] fine-tuned from ImageNet-21k-pretrained weights. All three models are trained under identical data preprocessing conditions and evaluated on the same held-out test split.

1.3 Research Objectives

The specific objectives of this project are as follows:

(1) Design and implement a custom CNN architecture to establish a baseline model.

(2) Fine-tune a pretrained ResNet-50 backbone on RAF-DB and analyze how residual learning and ImageNet transfer learning improve performance over the baseline CNN.

(3) Fine-tune a pretrained ViT-B/16 on RAF-DB and examine whether the global self-attention mechanism provides advantages to the local feature learning of CNNs.

(4) Quantitatively compare all three models across a set of evaluation metrics such as overall accuracy, macro-averaged F1 score, per-class precision and recall, ROC-AUC, and confusion matrices, with more focus on performance over minority emotion classes.

2 Related Work

Early FER systems relied on hand-crafted feature descriptors such as Local Binary Patterns (LBP), Histogram of Oriented Gradients (HOG), and Gabor filter banks, combined with Support Vector Machine (SVM) classifiers [7]. While these methods achieved strong accuracy on controlled laboratory datasets such as the Extended Cohn-Kanade Dataset (CK+) [8] and Japanese Female Facial Expression (JAFFE) [9], they were brittle to the pose, illumination, and occlusion variation present in real-world images and required substantial domain expertise to engineer.

The introduction of the FER-2013 benchmark by Goodfellow et al. [10] established CNNs as the standard approach for unconstrained FER. Subsequent work demonstrated that deeper architectures with improved training strategies consistently raised performance on large-scale benchmarks [3]. He et al. [5] introduced Residual Networks (ResNet), which resolved the degradation problem in deep networks through skip connections that provide direct gradient pathways during backpropagation. ResNet backbones pretrained on ImageNet became the dominant transfer learning foundation for FER. Huang et al. [11] showed that fine-tuning a pretrained ResNet on RAF-DB improved validation accuracy by approximately 27% over training from scratch, highlighting the value of ImageNet-pretrained features even when the target domain is face images.

More recently, the Vision Transformer (ViT) [6] has emerged as a strong alternative to CNNs for FER. Unlike CNNs, ViT captures long-range dependencies across facial regions from the first layer, without relying on local spatial inductive biases. Studies on RAF-DB have reported competitive accuracy from ViT-based models, with results ranging from 88.62% [12] to 91.10% [13], demonstrating that the Transformer architecture can match or exceed CNN-based approaches on this benchmark.

3 Dataset Description

3.1 The RAF-DB Dataset

The Real-world Affective Faces Database (RAF-DB), introduced by Li et al. [4], is a large-scale facial expression dataset comprising 15,339 images collected via internet search engines and annotated through a rigorous crowd-sourcing protocol. Each image was independently labeled by approximately 40 annotators, and the final emotion label was determined by majority vote, making the annotations substantially more reliable than those of comparable datasets collected with a single annotator per image. The single-label subset used in this work covers seven basic emotion categories: angry, disgust, fear, happy, neutral, sad, and surprise. It has a split of 12,271 training and 3,068 test images.

The dataset exhibits considerable real-world variability across subject age, ethnicity, head pose, lighting, and partial occlusions from glasses and facial hair. This ecological validity makes RAF-DB a more challenging and representative benchmark than laboratory-controlled datasets, where expressions are posed under uniform conditions. Sample images from each class are shown in Figure 1.

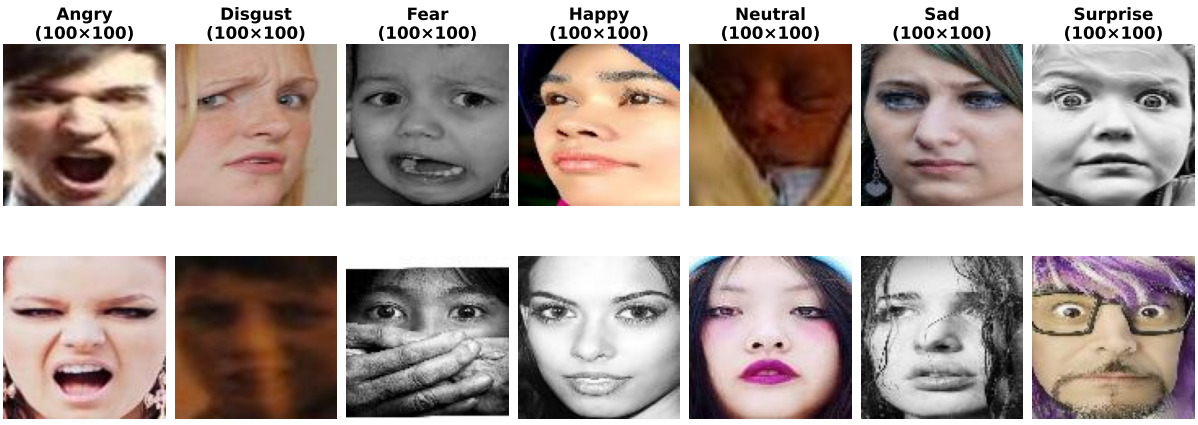

Figure 1: Sample images from the RAF-DB training set, two per emotion class.

RAF-DB was selected over the more commonly used FER-2013 benchmark for three reasons directly relevant to this project. First, all images are pre-aligned and uniformly sized to 100×100 pixels in RGB format, eliminating the need for grayscale-to-color channel replication required by FER-2013's 48×48 grayscale images. Second, the higher-quality crowd-sourced annotations reduce the label noise known to affect FER-2013 (estimated at $\sim$30% of images), providing a cleaner training signal. Third, the native color format and larger spatial resolution make RAF-DB inherently better suited to fine-tuning pretrained models such as ResNet-50 and ViT-B/16, which expect RGB inputs at 224×224 pixels.

3.2 Class Distribution and Imbalance Analysis

As shown in Figure 2, the dataset is severely imbalanced since *happy* accounts for 38.9% of training images (4,772 samples) while *fear* accounts for only 2.3% (281 samples), yielding an imbalance ratio of approximately 17×.

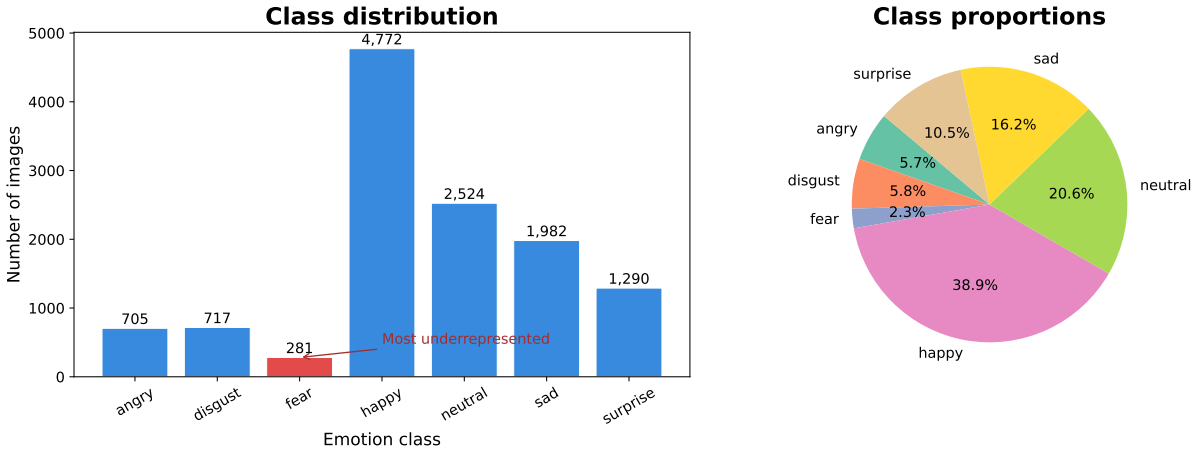

Figure 2: Class distribution of the RAF-DB training set. The *fear* class is the most underrepresented with 281 images, while *happy* dominates with 4,772 images.

A naive classifier predicting *happy* for every image would achieve 38.9% test accuracy without learning any discriminative features. Overall accuracy therefore provides a misleading picture of model quality under this distribution. To mitigate the effect of this imbalance during training, a class-weighted cross-entropy loss is used, assigning a penalty weight $w_c = N/(C \cdot n_c)$ to each class c, where N is the total number of training samples, $C = 7$ (number of classes), and n_c is the class count, so that misclassifications of underrepresented classes contribute more to the loss.

3.3 Data Preprocessing

All preprocessing is applied consistently across training and test sets. Images are resized from their native 100×100 resolution to 224×224 pixels using bilinear interpolation, to satisfy the input requirements of the pretrained ResNet-50 and ViT-B/16 architectures. Pixel values are normalized using per-channel mean and standard deviation statistics computed exclusively from the training set, yielding $\boldsymbol{\mu} = [0.575, 0.450, 0.401]$ and $\boldsymbol{\sigma} = [0.206, 0.189, 0.180]$. Dataset-specific statistics are used rather than ImageNet values ($\boldsymbol{\mu}_{\text{IN}} = [0.485, 0.456, 0.406]$, $\boldsymbol{\sigma}_{\text{IN}} = [0.229, 0.224, 0.225]$) because RAF-DB's face-dominated images exhibit a noticeably higher red-channel mean (0.575 vs. 0.485), reflecting the prevalence of skin tones. The 12,271-image training set is split 90/10 using a fixed random seed of 42, yielding 11,044 training and 1,227 validation images. The 3,068-image test split is used only for final evaluation.

3.4 Data Augmentation

Data augmentation is applied only during training. The pipeline consists of five transforms: *RandomHorizontalFlip* ($p = 0.5$) exploits the bilateral symmetry of the human face, providing a genuinely distinct training example. *RandomRotation* ($\pm 12°$) simulates natural head tilt variation present in real-world photographs. *RandomCrop* (size 224, padding 10, reflect mode) simulates small positional shifts without risking removal of discriminative regions such as the eyes or mouth. Aggressive cropping was avoided because RAF-DB images are already tightly face-aligned. *ColorJitter* (brightness ± 0.25, contrast ± 0.25, saturation ± 0.15, hue ± 0.05) reflects real-world illumination variation inherent in internet-collected photographs. Lastly, *RandomGrayscale* ($p = 0.1$) prevents the model from learning spurious color-based shortcuts, encouraging reliance on facial geometry.

4 Methodology

4.1 Overview of Architectural Comparison

Three models are implemented and compared, with each representing a distinct deep learning paradigm. The custom CNN establishes what is achievable from random initialization (e.g., He initialization) with a simple architecture. ResNet-50 and ViT-B/16 are fine-tuned from pretrained weights, allowing us to isolate the contribution of residual learning and global self-attention, respectively. Importantly, all three models are trained with the same loss function, optimizer, and data pipeline, so performance differences are attributable to architecture alone.

4.2 Model 1: Custom Convolutional Neural Network

The custom CNN consists of four convolutional blocks, each following the pattern:

$$\text{Conv2d}(3 \times 3, \text{padding} = 1) \rightarrow \text{BatchNorm2d} \rightarrow \text{ReLU} \rightarrow \text{MaxPool2d}(2 \times 2).$$

Filter counts double with each block (32, 64, 128, 256) progressively widening the feature representation while halving spatial resolution. Batch normalization stabilizes training by reducing internal covariate shift, and the 3×3 kernel size follows the principle that stacked small kernels provide equivalent receptive field to larger kernels with fewer parameters [14].

After the four convolutional blocks, Global Average Pooling (GAP) replaces flattening, reducing each feature map to a scalar and making the network invariant to small spatial translations. The GAP output feeds into a fully connected layer of 256 units with ReLU activation and Dropout(0.5), followed by the 7-class output layer. All weights are initialized using He initialization, which is appropriate for ReLU activations since it sets the variance of the random weight distribution to $\sigma^2 = 2/n_{\text{in}}$, where n_{in} is the number of

input units to each layer, preventing vanishing or exploding activations at the start of training. The total parameter count is approximately 500K. This small size is intentional because the custom CNN is a compute-efficient baseline, not a state-of-the-art model. The architecture is summarized in Figure 3.

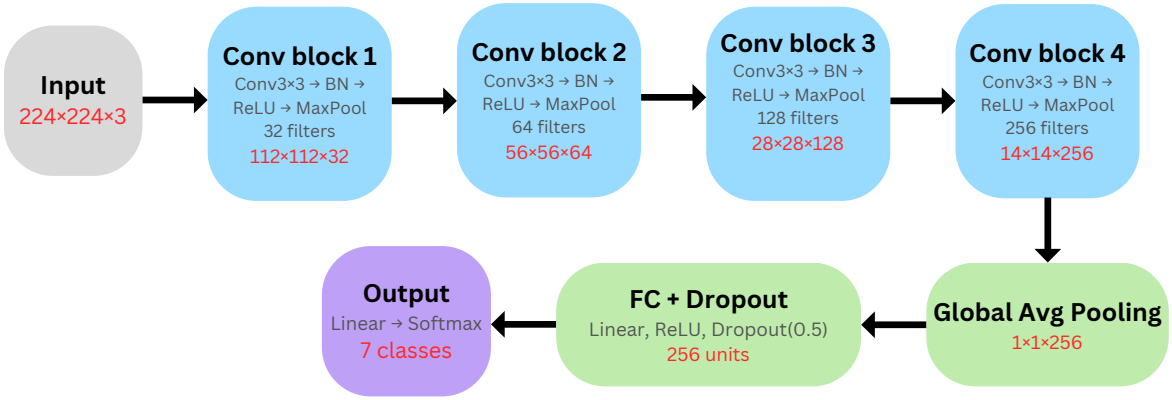

Figure 3: Custom CNN architecture. The network processes a $224 \times 224 \times 3$ input through four convolutional blocks of increasing depth (32, 64, 128, 256 filters), each applying Conv3 $\times$ 3, batch normalization, ReLU, and MaxPool. GAP collapses spatial dimensions, and a fully connected layer with Dropout(0.5) feeds the 7-class softmax output.

4.3 Model 2: ResNet-50 with Transfer Learning

ResNet-50 [5] addresses the degradation problem in deep networks by introducing skip connections. Rather than learning the full target mapping $\mathcal{H}(\mathbf{x})$ directly, each residual block learns the residual $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$, and the block output is $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$. These connections provide a direct gradient pathway during backpropagation, alleviating vanishing gradients and enabling stable training at 50 layers. The four residual stages contain 3, 4, 6, and 3 bottleneck blocks respectively, each consisting of a 1×1 convolution for channel reduction, a 3×3 convolution for spatial feature extraction, and a 1×1 convolution for channel restoration. The full architecture and feature map dimensions at each stage are illustrated in Figure 4.

The model is loaded with `IMAGENET1K_V2` pretrained weights via `torchvision`. The $224 \times 224 \times 3$ input passes through the frozen stem (a 7×7 convolution with stride 2 followed by batch normalization, ReLU, and max pooling), reducing the spatial resolution to $56 \times 56 \times 64$. Layers 1, 2, and 3 then progressively downsample and widen the feature maps, all with frozen weights. The output of Layer 3 enters the trainable Layer 4, which produces a $7 \times 7 \times 2048$ feature map. A layer-wise freezing strategy is applied such that the stem and the first three residual stages (Layers 1–3) are frozen at their ImageNet-learned weights, retaining the edge detectors and texture filters that transfer directly to face images. Only Layer 4 and the new classification head are trained on RAF-DB. The original

1000-class head is replaced with a linear layer mapping the 2048-dimensional output of global average pooling to 7 emotion classes. This leaves approximately 8.5M parameters frozen and 15M trainable, preventing overfitting on 11,044 training images while allowing task-specific adaptation in the deepest residual stage.

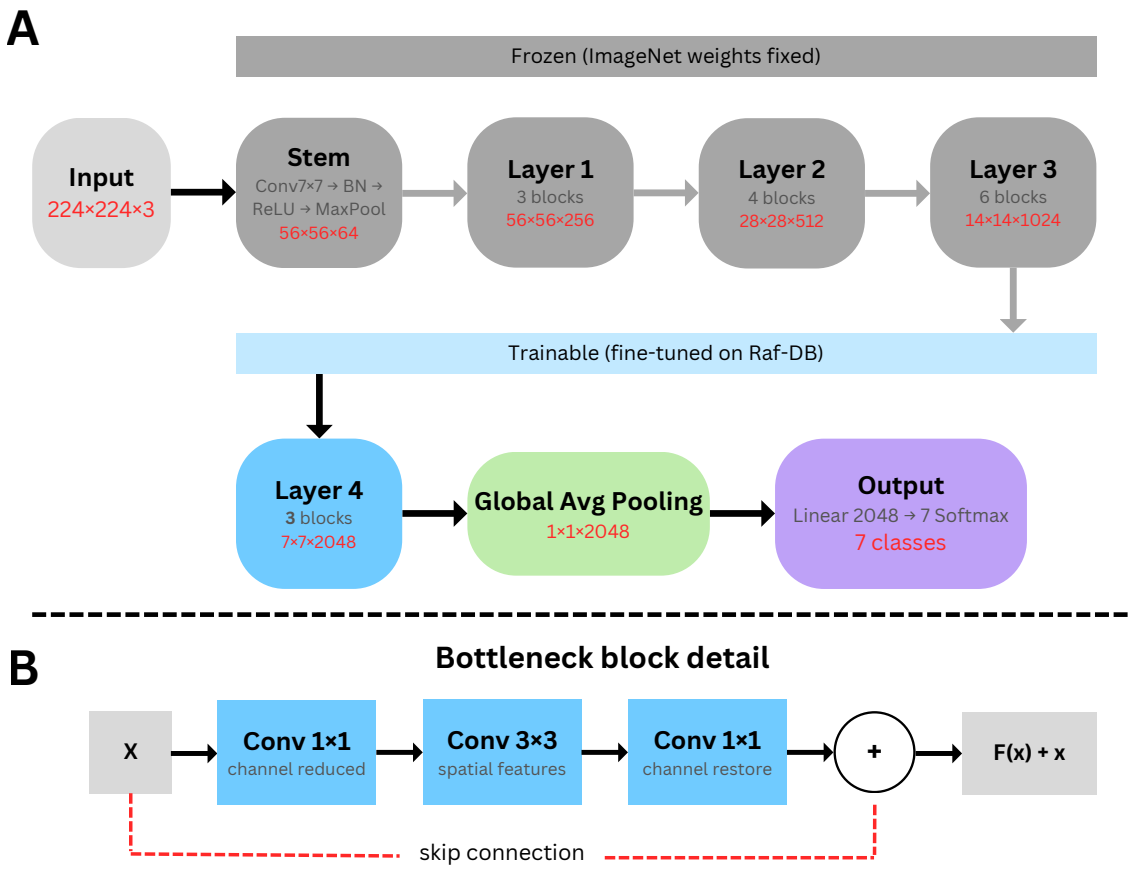

Figure 4: **(A)** ResNet-50 fine-tuning strategy: gray blocks are frozen at their ImageNet-pretrained weights, remaining blocks are trained on RAF-DB. Feature map dimensions show the progressive spatial downsampling ($224 \times 224 \rightarrow 7 \times 7$) and channel expansion ($3 \rightarrow 2048$) through the network. **(B)** Bottleneck block detail: the skip connection bypasses the three convolutions, providing a direct gradient pathway during backpropagation.

4.4 Model 3: Vision Transformer (ViT-B/16) with Transfer Learning

ViT-B/16 [6] processes images as sequences of patch tokens. The 224×224 input is divided into a 14×14 grid of non-overlapping 16×16 patches, yielding 196 tokens. Each patch is linearly projected into a 768-dimensional embedding. A learnable `[CLS]` token is prepended and positional encodings are added, producing a sequence of 197 tokens that is passed through 12 Transformer encoder blocks, each with 12 attention heads. At every block, the multi-head self-attention computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{1}$$

where Q, K, V are query, key, and value projections of each token and $d_k = 64$ is the per-head dimension. Unlike CNNs, which build spatial context hierarchically through stacked local convolutions, every token attends to every other token from the first block, providing a global receptive field throughout the network. Each block also contains a position-wise MLP ($768 \to 3072 \to 768$ dimensions) and two residual connections with layer normalization, one wrapping the attention sub-layer and one wrapping the MLP, as illustrated in Figure 5.

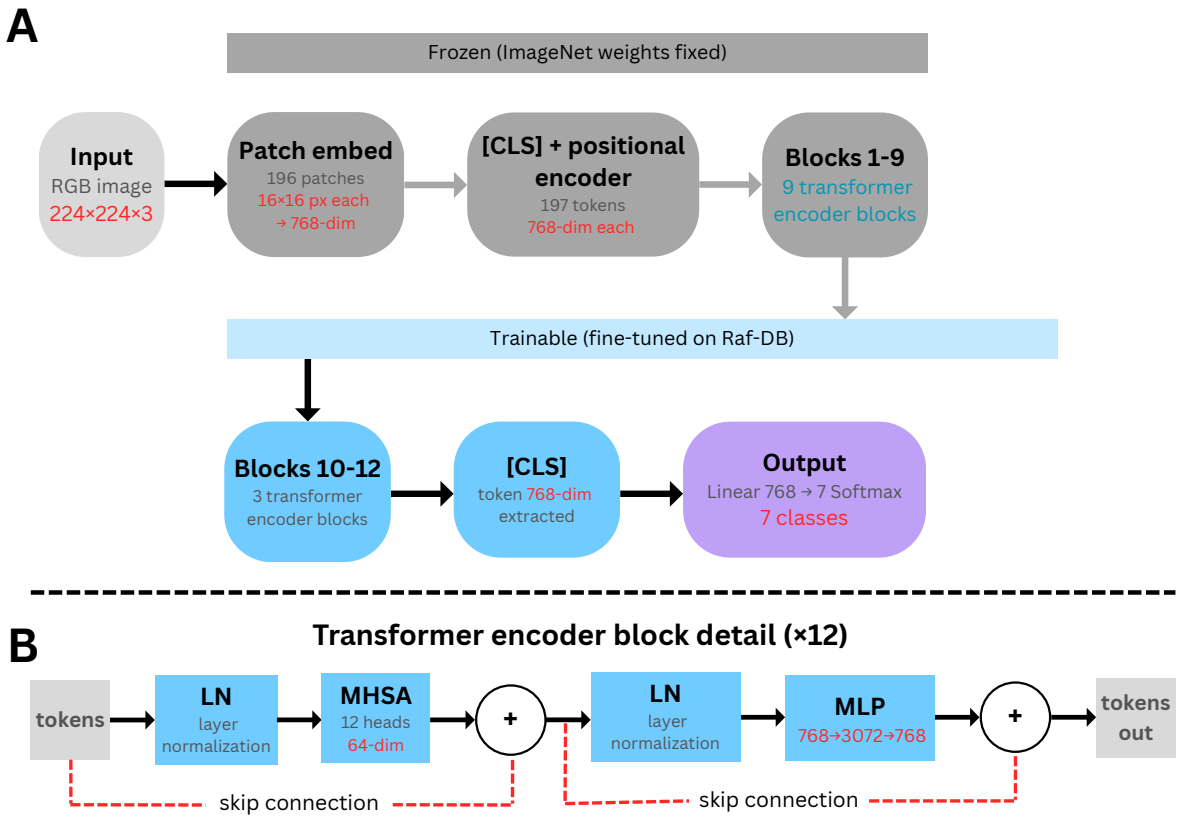

Figure 5: **(A)** ViT-B/16 fine-tuning strategy: the input is split into 196 patches of 16×16 pixels, each embedded into 768 dimensions. A prepended `[CLS]` token and positional encodings produce a sequence of 197 tokens processed by 12 encoder blocks. Blocks 1–9 are frozen at their ImageNet-21k weights, blocks 10–12 and the new classification head are trained on RAF-DB. **(B)** The block detail multi-head self-attention (MHSA) followed by a feed-forward MLP wrapped with a residual connection.

The model is loaded with ImageNet-21k pretrained weights via the `ViTForImageClassification` interface. Blocks 1–9 are frozen and only blocks 10–12 along with the new classification head are fine-tuned. The pretrained head is replaced with a linear layer mapping the 768-dimensional `[CLS]` token representation to 7 emotion classes. Differential learning

rates are applied: the fine-tuned transformer blocks use a lower rate than the new head, preventing catastrophic forgetting of pretrained representations. This leaves approximately 64M parameters frozen and 22M trainable.

4.5 Loss Function

To address the 17× class imbalance in RAF-DB, class-weighted cross-entropy is used:

$$\mathcal{L} = -\sum_{c=1}^{C} w_c \, y_c \log \hat{p}_c, \qquad w_c = \frac{N}{C \cdot n_c}, \tag{2}$$

where N is the total number of training samples, $C = 7$, n_c is the count of class c, $\hat{p}_c$ is the predicted probability, and $y_c \in \{0, 1\}$ is the hard label. A misclassification of a *fear* image therefore contributes approximately 17 times more to the loss than a *happy* misclassification.

5 Experimental Setup

5.1 Implementation and Hardware

All models are implemented in Python using PyTorch [15], with pretrained weights accessed via `torchvision` (ResNet-50) and the HuggingFace `transformers` library (ViT-B/16). Training is performed on Google Colab using an NVIDIA A100 GPU (40 GB VRAM). A global random seed of 42 is applied to Python's `random` module, NumPy, PyTorch (CPU and GPU), and all CUDA operations (`cudnn.deterministic = True`, `benchmark = False`), with DataLoader workers seeded via a custom `seed_worker` function. This ensures fully reproducible results across runs.

5.2 Training Configuration

All three models share the same training loop structure. Each epoch consists of a training phase (model in `train()` mode) and a validation phase (`eval()` mode, no gradient computation). The Adam optimizer is used with a batch size of 32. A `ReduceLROnPlateau` scheduler monitors validation loss and reduces the learning rate by a factor of 0.5 when no improvement is observed for 5 consecutive epochs. Early stopping halts training if validation macro-F1 does not improve for 10 epochs, and the checkpoint with the best validation macro-F1 is restored for final evaluation. Model-specific hyperparameters are reported in Table 1. For ViT-B/16, the learning rate of 1e-5 applies to the fine-tuned transformer blocks (10–12) and 1e-4 to the new classification head, implementing the differential learning rate strategy described in Section 4.4.

Table 1: Hyperparameter configuration and model complexity for each model.

Property	**Custom CNN**	**ResNet-50**	**ViT-B/16**
Architecture			
Total parameters	456K	23.5M	85.8M
Trainable parameters	456K	15.0M	21.3M
Pretraining data	None	ImageNet (1.2M)	ImageNet-21k (14M)
Training			
Max epochs	100	100	100
Batch size	32	32	32
Optimizer	Adam	Adam	Adam
Learning rate	1e-3	1e-4	1e-5 / 1e-4
LR scheduler		ReduceLROnPlateau	
Scheduler factor	0.5	0.5	0.5
Scheduler patience	5	5	5
Weight decay	1e-4	1e-4	1e-4
Dropout	0.5	—	—
Early stopping patience	10	10	10

5.3 Evaluation Metrics

Given the 17× class imbalance in RAF-DB, macro-averaged F1 score is used as the primary evaluation metric. It computes F1 independently for each class and averages them equally:

$$\text{Macro-F1} = \frac{1}{C}\sum_{c=1}^{C}\frac{2 \cdot P_c \cdot R_c}{P_c + R_c}, \tag{3}$$

where precision and recall for class c are defined as

$$P_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c}, \qquad R_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c}, \tag{4}$$

and overall accuracy is $\text{Acc} = \sum_c \text{TP}_c / N$, with N the total number of test samples. Unlike overall accuracy, macro-F1 treats every class equally regardless of frequency, making it the honest measure of performance under imbalance. Per-class one-vs-rest ROC-AUC is also reported, defined as the area under the curve of the true positive rate $\text{TPR}_c = \text{TP}_c/(\text{TP}_c + \text{FN}_c)$ against the false positive rate $\text{FPR}_c = \text{FP}_c/(\text{FP}_c + \text{TN}_c)$ as the decision threshold varies. Confusion matrices and training/validation loss curves are included as supplementary diagnostics.

6 Results

6.1 Training Dynamics

Figure 6 shows the training and validation loss curves for all three models. The custom CNN decreases loss slowly with considerable oscillation over 97 epochs, with training and validation loss remaining closely tracked throughout, consistent with underfitting from random initialization on a limited dataset. ResNet-50 drops training loss sharply in the first ten epochs, while validation loss plateaus around 1.4 and diverges progressively, reflecting the tension between frozen ImageNet features and task-specific adaptation in Layer 4. The best validation macro-F1 for ResNet-50 is reached at epoch 40, before early stopping fires at epoch 50. ViT-B/16 converges fastest, with both losses falling steeply in the first five epochs and validation loss stabilizing around 0.95 from epoch 10 with minimal divergence, suggesting that the pretrained global attention representations require only minor adjustment for face-aligned inputs. The best macro-F1 for ViT-B/16 is at epoch 26, and early stopping fires at epoch 36.

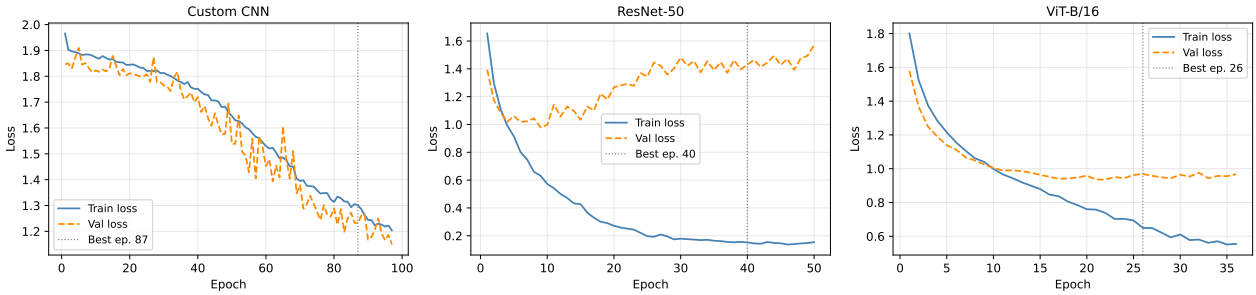

Figure 6: Training and validation loss curves. The dotted vertical line marks the epoch with the best validation macro-F1.

6.2 Overall Performance Comparison

Table 2 reports validation and test-set performance across all metrics. ResNet-50 achieves the highest test values on all metrics (accuracy 77.18%, macro-F1 0.6939), followed by ViT-B/16 (70.96%, 0.6041) and the custom CNN (61.54%, 0.5217), with accuracy and macro-F1 rankings agreeing across all three models.

Table 2: Validation and test set performance comparison.

	Validation	**Test**			
Model	**Macro-F1**	**Accuracy**	**Macro-F1**	**Macro-Precision**	**Macro-Recall**
Custom CNN	0.5184	61.54%	0.5217	0.5241	0.5335
ResNet-50	0.6862	77.18%	0.6939	0.6938	0.6969
ViT-B/16	0.6517	70.96%	0.6041	0.5865	0.6333

The validation macro-F1 scores reveal additional differences in generalization behavior. The custom CNN shows near-identical val and test macro-F1 (0.5184 vs. 0.5217). ResNet-50 shows a small positive shift from validation to test (0.6862 vs. 0.6939). ViT-B/16, by contrast, shows the largest val-to-test drop (0.6517 vs. 0.6041), the only model where test performance falls meaningfully below validation.

6.3 Per-Class Performance Analysis

Table 3 reports per-class F1 scores. ResNet-50 achieves the highest F1 in every class. *Happy* is the best-recognized class across all models (F1: 0.7738, 0.8842, 0.8482). *Disgust* is the lowest-scoring class across all models (F1: 0.2518, 0.4713, 0.3691), followed by *sad* for the custom CNN (0.3858) and *fear* for ViT-B/16 (0.3731). These results highlight that minority classes (disgust and fear) remain the most difficult across all architectures.

Table 3: Per-class F1 score for each model on the test set.

Emotion	Custom CNN	ResNet-50	ViT-B/16
Angry	0.4959	0.7104	0.5850
Disgust	0.2518	0.4713	0.3691
Fear	0.4930	0.5588	0.3731
Happy	0.7738	0.8842	0.8482
Neutral	0.6117	0.7218	0.6848
Sad	0.3858	0.7348	0.6432
Surprise	0.6402	0.7763	0.7255

6.4 Confusion Matrices

Figure 7 shows that *disgust* is most frequently predicted as *happy* (CNN: 0.17, ResNet: 0.21, ViT: 0.22) and *neutral* (CNN: 0.16, ResNet: 0.21, ViT: 0.22). *Fear* is most commonly predicted as *surprise* across all models (CNN: 0.20, ResNet: 0.20, ViT: 0.16). *Sad* is consistently confused with *neutral*. ResNet-50 shows the darkest diagonal overall.

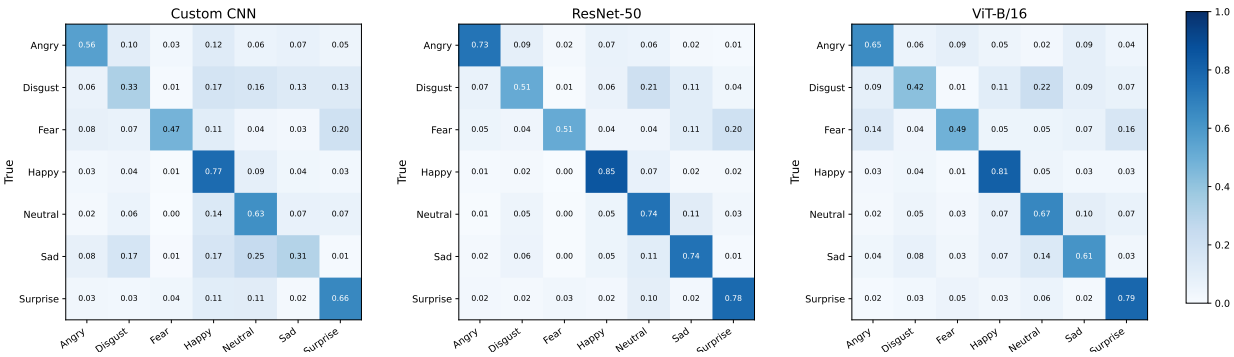

Figure 7: Normalized confusion matrices on the test set.

6.5 ROC Curves and AUC Scores

Table 4 reports per-class and macro-average one-vs-rest AUC scores; the corresponding ROC curves are shown in Figure 10 (Appendix A.3). The macro-AUC ranking mirrors the macro-F1 ranking: ResNet-50 leads (0.944), followed by ViT-B/16 (0.928) and the custom CNN (0.872). At the class level, *disgust* yields the lowest per-class AUC across all models (0.766, 0.897, 0.887), consistent with its position as the lowest-scoring class in Table 3. *Happy* and *surprise* are the most discriminable classes across all models, with ResNet-50 reaching AUC 0.969 for both.

Table 4: One-vs-rest ROC-AUC scores.

Emotion	**Custom CNN**	**ResNet-50**	**ViT-B/16**
Angry	0.910	0.958	0.939
Disgust	0.766	0.897	0.887
Fear	0.906	0.928	0.911
Happy	0.898	0.969	0.956
Neutral	0.871	0.937	0.921
Sad	0.829	0.951	0.919
Surprise	0.922	0.969	0.964
Macro AUC	0.872	0.944	0.928

7 Discussion

ResNet-50's advantage over the custom CNN (macro-F1: 0.6939 vs. 0.5217) reflects two compounding factors: the depth of a 50-layer residual network versus a four-block baseline, and ImageNet pretraining that supplies edge detectors and texture filters directly transferable to face images. The CNN's slow, oscillatory convergence over 97 epochs with closely tracked train-val loss, and its near-identical val and test macro-F1, confirm underfitting throughout training. With only 11,044 images, learning discriminative facial features from random initialization at this model scale is insufficient.

The gap between ResNet-50 and ViT-B/16 (macro-F1: 0.6939 vs. 0.6041) is attributable to dataset scale and inductive bias. ViT's global self-attention offers richer representational capacity in principle, but lacks the spatial inductive bias of convolutions, specifically translation equivariance and locality, that makes ResNet-50 naturally suited to the tightly face-aligned RAF-DB images. With only 11,044 fine-tuning images, ViT cannot learn sufficient spatial structure to compensate, consistent with the established data requirements of transformer models [6]. The largest val-to-test macro-F1 drop observed for ViT-B/16 (0.6517 vs. 0.6041) further supports this, suggesting the model overfit to the validation

distribution despite early stopping.

Disgust is the most challenging class across all models despite having 717 training samples, which is more than fear (281). The confusion matrices confirm that its difficulty stems from visual ambiguity with happy and neutral rather than from sample scarcity. The fear-surprise confusion is similarly interpretable: both expressions share raised eyebrows and widened eyes, making them difficult to separate from facial appearance alone. Class-weighted loss measurably benefits fear recognition across all models, but cannot resolve the intrinsic visual ambiguity of disgust, which remains the lowest-scoring class even under ResNet-50 (F1: 0.4713, AUC: 0.897).

RAF-DB's 15,339 images are relatively small for deep learning, particularly for transformer-based models. The 17× class imbalance and a validation set of only 1,227 images introduce noise in the macro-F1 estimates. Bilinear upscaling from 100×100 to 224×224 pixels may blur texture features critical for subtle emotion classes. Several directions could address these limitations. Adding generative oversampling of disgust and fear could provide a stronger remedy for class imbalance than class weighting alone. ViT-B/16 would benefit from cosine annealing and a larger proportion of unfrozen blocks to better adapt its attention patterns to facial structure.

8 Conclusion

This project implemented and compared three deep learning architectures for seven-class facial emotion recognition on RAF-DB under identical training conditions, so that performance differences are attributable to architectural choice alone. The custom CNN established a clear baseline at 61.54% accuracy and a macro-F1 of 0.5217, confirming that learning discriminative facial features from random initialization on a moderately sized dataset is insufficient at this model scale (Objective 1). Fine-tuning ResNet-50 with a layer-wise freezing strategy raised accuracy to 77.18% and macro-F1 to 0.6939, demonstrating that residual learning combined with ImageNet transfer substantially improves over the baseline (Objective 2). Fine-tuning ViT-B/16 yielded 70.96% accuracy and macro-F1 of 0.6041, trailing ResNet-50 on every metric and showing that global self-attention does not provide an advantage over convolutional feature learning at RAF-DB's scale (Objective 3). Across the full evaluation suite, ResNet-50 ranked first on every measure, including minority emotion classes, while disgust remained the hardest class for all models (best F1: 0.4713) despite an adequate sample count, pointing to intrinsic inter-class visual ambiguity that class-weighted loss alone cannot resolve (Objective 4). These findings confirm that for moderately sized and imbalanced facial expression datasets, pretrained convolutional networks remain the most effective architecture, with next steps including cross-dataset generalization and facial landmark auxiliary inputs.

References

[1] Paul Ekman and Wallace V Friesen. Constants across cultures in the face and emotion. *J. Pers. Soc. Psychol.*, 17(2):124, 1971.

[2] Manmeet Kaur and Munish Kumar. Facial emotion recognition: A review. *Expert Syst.*, 41(10):e13670, 2024.

[3] Shan Li and Weihong Deng. Deep facial expression recognition: A survey. *IEEE Trans. Affect. Comput.*, 13(3):1195–1215, 2020.

[4] Shan Li, Weihong Deng, and JunPing Du. Reliable crowdsourcing for expression recognition. In *Proc. CVPR*, pages 2852–2861, 2017.

[5] Kaiming He et al. Deep residual learning. In *Proc. CVPR*, pages 770–778, 2016.

[6] Alexey Dosovitskiy et al. An image is worth 16x16 words. *arXiv:2010.11929*, 2020.

[7] Thomas Kopalidis et al. Advances in facial expression recognition. *Information*, 15(3):135, 2024.

[8] Patrick Lucey et al. The extended cohn-kanade dataset (ck+). In *Proc. CVPR Workshops*, pages 94–101, 2010.

[9] Michael J Lyons et al. Coding facial expressions with gabor wavelets. *arXiv:2009.05938*, 2020.

[10] Ian J Goodfellow et al. Challenges in representation learning. In *Int. Conf. Neural Inf. Process.*, pages 117–124, 2013.

[11] Zi-Yu Huang et al. Computer vision for facial emotion recognition. *Sci. Rep.*, 13(1):8425, 2023.

[12] Hanting Li et al. Mask vision transformer for fer. *arXiv:2106.04520*, 2021.

[13] Shuyi Mao et al. Au-aware vision transformers for fer. *arXiv:2211.06609*, 2022.

[14] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks. *arXiv:1409.1556*, 2014.

[15] Adam Paszke et al. Pytorch: High-performance deep learning library. In *Adv. Neural Inf. Process. Syst. (NeurIPS)*, volume 32, 2019.

A Additional Results

A.1 Validation Macro-F1 per Epoch

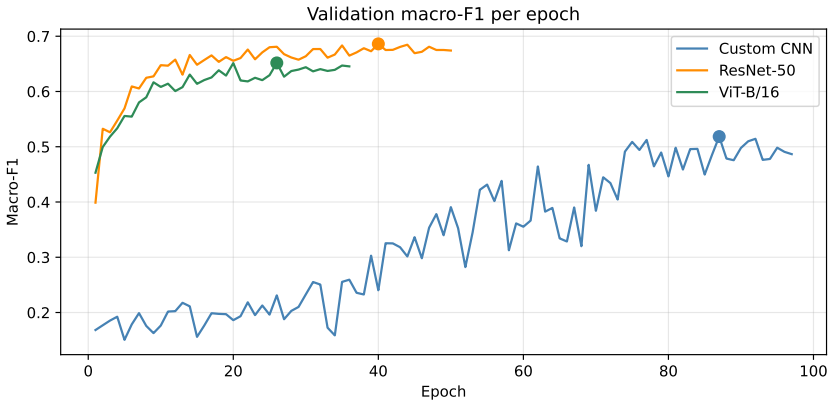

Figure 8: Validation macro-F1 per epoch for all three models. Filled circles mark the best checkpoint epoch for each model: epoch 87 for the custom CNN (macro-F1 = 0.5184), epoch 40 for ResNet-50 (0.6862), and epoch 26 for ViT-B/16 (0.6517).

A.2 Sample Predictions

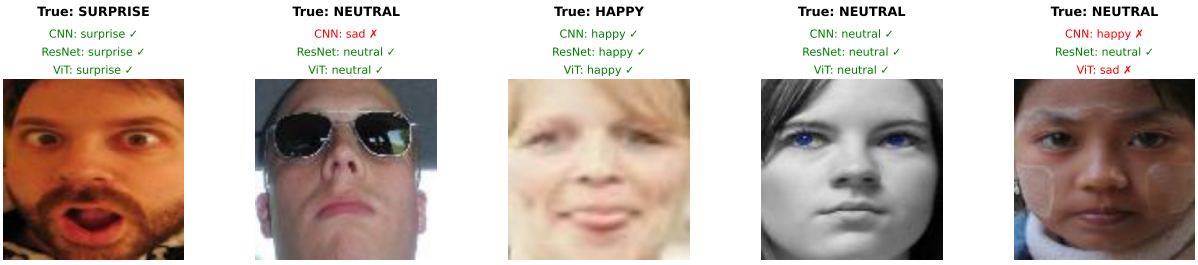

Figure 9: Sample test images with predicted labels from all three models.

A.3 ROC Curves

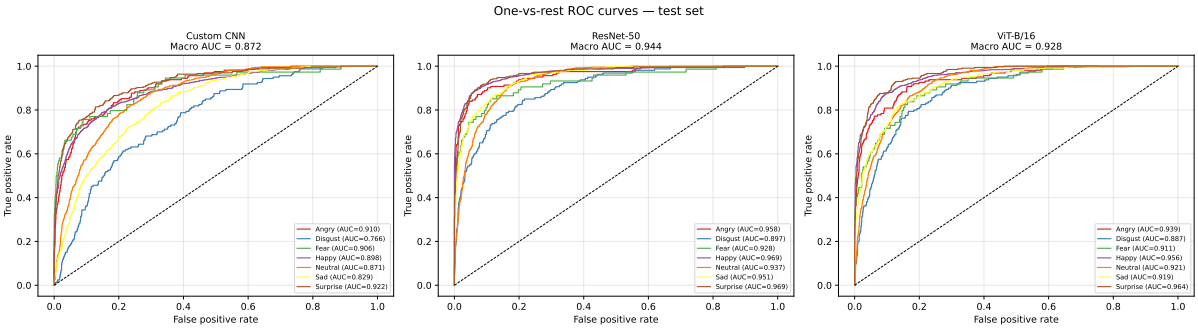

Figure 10: One-vs-rest ROC curves for all three models on the test set. Macro-average AUC is shown in each subplot title.

B Code Snippets

B.1 Custom CNN Architecture

Listing 1: Custom CNN architecture definition in PyTorch.

```
class ConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=3,
                      padding=1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2, 2)
        )
    def forward(self, x):
        return self.block(x)

class CustomCNN(nn.Module):
    def __init__(self, num_classes=7):
        super().__init__()
        self.features = nn.Sequential(
            ConvBlock(3,   32),   # 224x224 -> 112x112
            ConvBlock(32,  64),   # 112x112 ->  56x56
            ConvBlock(64,  128),  #  56x56  ->  28x28
            ConvBlock(128, 256),  #  28x28  ->  14x14
        )
        self.gap  = nn.AdaptiveAvgPool2d(1)
        self.head = nn.Sequential(
            nn.Flatten(),
            nn.Linear(256, 256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(256, num_classes)
        )
# Total parameters: ~456K  |  All trainable (no pretraining)
```

B.2 ResNet-50 Fine-Tuning Setup

Listing 2: ResNet-50 layer-wise freezing strategy and 7-class head replacement.

```
# Freeze stem and layers 1-3 (~8.5M parameters)
frozen_modules = [
    resnet_model.conv1, resnet_model.bn1,
    resnet_model.maxpool, resnet_model.layer1,
    resnet_model.layer2, resnet_model.layer3
]
```

```
for module in frozen_modules:
    for param in module.parameters():
        param.requires_grad = False

# Replace 1000-class head with 7-class head
resnet_model.fc = nn.Linear(
    resnet_model.fc.in_features, NUM_CLASSES
)
# Total: ~23.5M  |  Trainable (Layer 4 + head): ~15.0M
```

B.3 ViT-B/16 Differential Learning Rate Setup

Listing 3: ViT-B/16 selective unfreezing and differential learning rates for blocks 10–12 and the classification head.

```
# Freeze all, then unfreeze blocks 10-12 and layer norm
for param in vit_model.parameters():
    param.requires_grad = False

for block_idx in [9, 10, 11]:
    for param in vit_model.vit.encoder.layer[
            block_idx].parameters():
        param.requires_grad = True

for param in vit_model.vit.layernorm.parameters():
    param.requires_grad = True

# Reinitialize and unfreeze classification head
for param in vit_model.classifier.parameters():
    param.requires_grad = True

# Differential learning rates:
# fine-tuned blocks + layernorm at 1e-5, new head at 1e-4
vit_block_params = [
    p for i in [9, 10, 11]
    for p in vit_model.vit.encoder.layer[i].parameters()
]
optimizer = torch.optim.Adam([
    {'params': vit_block_params +
               list(vit_model.vit.layernorm.parameters()),
     'lr': 1e-5},
    {'params': vit_model.classifier.parameters(),
     'lr': 1e-4}
], weight_decay=1e-4)
# Total: ~85.8M  |  Trainable (blocks 10-12 + head): ~21.3M
```